

LogPulse AI - Incident Report

Generated : 2026-05-29 17:11:04 UTC

Trace ID : b2ab9379-c12f-48d7-b99a-e9683601541a

1 - Incident Summary

Incident ID : b2ab9379
Category : Memory
Severity : SeverityLevel.CRITICAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

17:10:34 ERROR worker-api memory usage critical: 94% of 256Mi limit, GC thrashing

3 - Classification

Category : Memory
Severity : SeverityLevel.CRITICAL
Summary : Worker API application exceeds its memory limit, experiencing severe garbage collection thrashing.
Source : Kubernetes

4 - Diagnosis

Root Cause : Based on the matched playbook 'Container OOMKilled Despite Low Reported Memory Usage', the likely root cause is a Memory incident matching the pattern: Worker API application exceeds its memory limit, experiencing severe garbage collection thrashing..
Confidence : 53%
Reasoning : Gemini unavailable; diagnosis derived from playbook 'Container OOMKilled Despite Low Reported Memory Usage' (RAG confidence=0.5370).
Source Tier : playbook_fallback
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Container OOMKilled Despite Low Reported Memory Usage' (RAG confidence=0.5370). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. SSH to the node running the problematic pod.
- 2. Inspect raw cgroup memory usage for the container: ``cat /sys/fs/cgroup/memory/kubepods/.../memory.usage_in_bytes`` (adjust path). Compare with ``memory.limit_in_bytes``.
- 3. Check ``dmesg -T | grep 'Out of memory'`` for the specific process killed.
- 4. Temporarily increase the container's memory ``limit`` to see if the OOM kill ceases.
- 5. Investigate if the application is creating large amounts of shared memory or ``tmpfs`` mounts, which are part of cgroup memory but not always in ``RSS``.
- 6. Consider upgrading the node kernel, ``kubelet``, or container runtime if known bugs related to cgroup accounting exist for the current versions.

Commands:

```
$ SSH to the node running the problematic pod.  
$ dmesg -T | grep 'Out of memory'  
$ kubectl patch deployment worker-api -p  
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}}}}'
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.SAFE**

Safe Commands (3):

- [OK] SSH to the node running the problematic pod.
- [OK] `dmesg -T | grep 'Out of memory'`
- [OK] `kubectl patch deployment worker-api -p '{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}}}}'`

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-29T14:11:03.334351+00:00

Sandbox Results:

Command : SSH to the node running the problematic pod.

Exit : 0

Time : 0ms

Output:

```
[auto-approve] simulated: SSH to the node running the problematic pod.
```

Command : `dmesg -T | grep 'Out of memory'`

Exit : 0

Time : 0ms

Output:

[auto-approve] simulated: dmesg -T | grep 'Out of memory'

Command : kubectl patch deployment worker-api -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}]}}}'

Exit : 0

Time : 0ms

Output:
[auto-approve] simulated: kubectl patch deployment worker-api -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}]}}}'

8 - Historical Context

Times Seen : 1

Past Outcomes:
- analyzed on 2026-05-29 13:56:07.805777+00:00

9 - Incident Timeline

[14:11:03] WORKFLOW_COMPLETED -> Completed in 28575ms